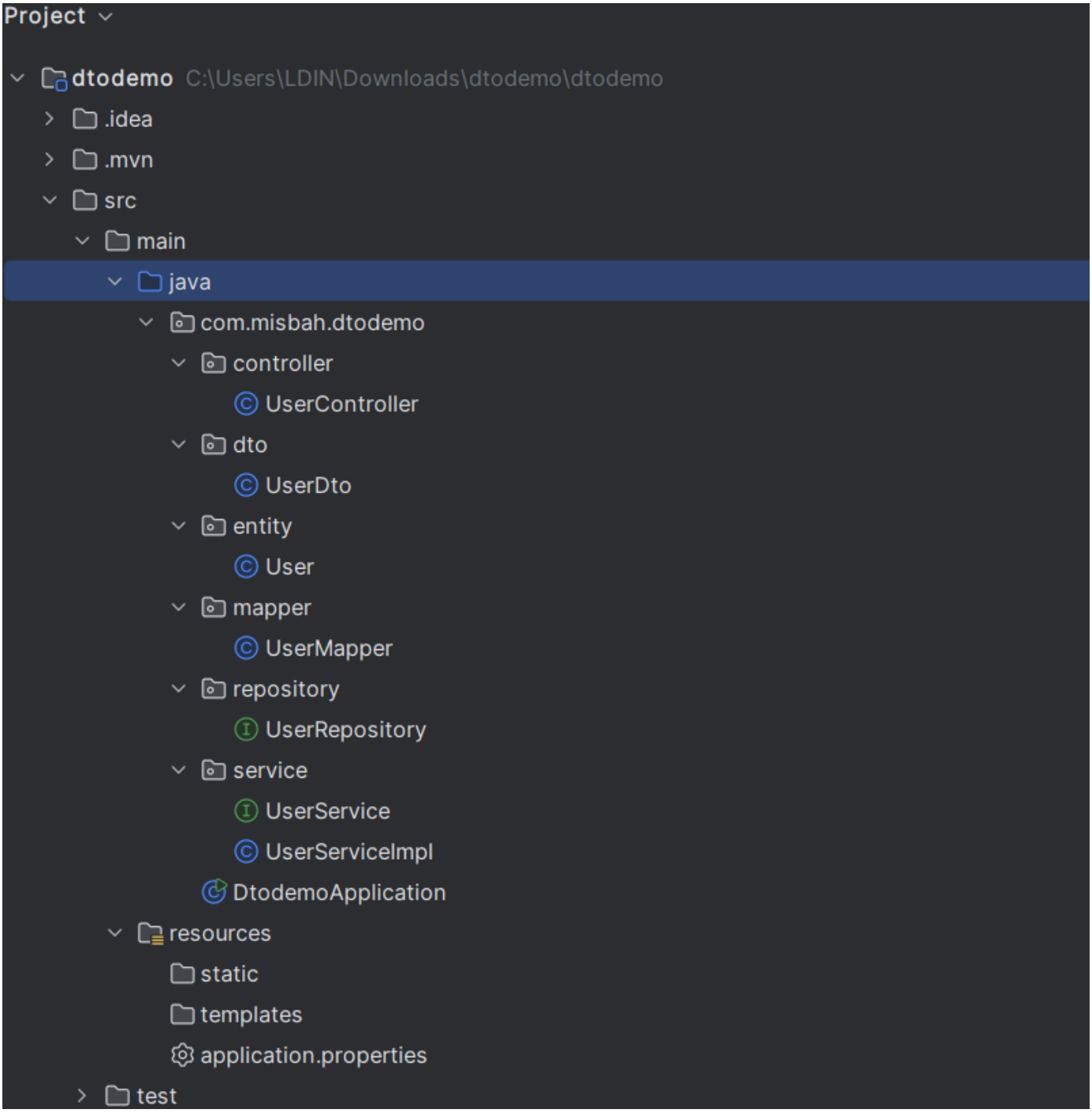
## SPRING PROJECT DIRECTORY




**Entity** : An Entity class represents a real world object like a User ,Book, Task, Product and maps to a table in the Database

An Entity is a Java class annotated by @Entity

Used by Spring Data JPA to know how to store and retrieve data in your database

Theres a Table called user in DB ,It has columns id,name,email,password

Each User object is one row

```java
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String password;
}
```

Client (Frontend or API tool)

|

[Controller Layer]

|

[Service Layer]

|

---

| ENTITY |

| (e.g., User.java) |

---

|

[Repository Layer]

|

Actual Database (PostgreSQL, etc.)

**Who all use User Entity and Why**

**Repository Layer** :It directly uses User Entity

Tells Spring Data JPA to save User objects into the actual user table in the database

This is where the entity -> DB connection happens

```
public interface UserRepository extends
JpaRepository<User,Long> {
}
```

**Service Layer** : responsible for performing core  operations ,applying rules and handling processes -> It contains the business logic that works with entities when saving ,updating ,deleting

It may convert DTOs to entities before saving

**Controller Layer :**

Controllers mostly work with DTOs ,they call the Service Layer that builds the User entity to store

**DTO Mappers:**

This is the layer that connects Entity to Dtos ,keeps API responses clean while storing full info in DB

**Service layer** :

It is responsible for performing core operations applying rules and handling process that make app do meaningful tasks

Controller handles POST /api/users -> To create new user

But controller should not directly be given access to save to DB or Calculate Logic(like limits ,validations rules etc.) or work with raw entity objects

Service Layer does the following

Converts DTOs → Entity

Applies business Logic

Calls Repository to save or retrieve

Converts Entity →DTO

Returns Safe and meaningful results to controller

**Relationship Between UserService and UserServiceImpl**

in typical Spring Boot apps

UserService -> Interface

```
package com.misbah.dtodemo.service;

import com.misbah.dtodemo.dto.UserDto;
import java.util.List;
public interface UserService
{
    UserDto createUser(UserDto userDto,String password);
    List<UserDto> getAllUsers();

}
```

This interface defines contract ->list what services are available

UserServiceImpl -> Class that implements the interface

```
public class UserServiceImpl implements UserService
{
    private final UserRepository userRepository;

    public UserServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDto createUser(UserDto userDto, String
password) {
        User user= UserMapper.toEntity(userDto,password);
        return UserMapper.toDto(userRepository.save(user));
    }

    @Override
    public List<UserDto> getAllUsers() {
        return userRepository.findAll()
                .stream()
                .map(UserMapper::toDto)
                .toList();
    }
}
```

The UserService interface defines what operations must be available like (createUser ,getAllUsers ) ,This class provides how those operations actually work

private final UserRepository userRepository;

This is a dependency , It gives access to CRUD operations for User entities via JpaRepository , we need an instance of this class for making our Service be able to save users into DB .Follows Dependency Injection principle

public UserServiceImpl(UserRepository userRepository) {

this.userRepository = userRepository;

} This is the constructor , Spring Boot will automatically call this and pass in an instance of UserRepository

We are using Constructor injection to do DI in Spring Boot

```
@Override
public UserDto createUser(UserDto userDto, String password) {
    User user= UserMapper.toEntity(userDto,password);
    return UserMapper.toDto(userRepository.save(user));
}

@Override
public List<UserDto> getAllUsers() {
    return userRepository.findAll()
            .stream()
            .map(UserMapper::toDto)
            .toList();
}
```

We are implementing both the methods defined in userService interface ,it takes a UserDto (public data from API ) and password

so when our controller gets a post request to create object it talks to service layer that has access to DB via the Repository object and hence uses it to save

,when we get a GET request from our controller ,the controller then uses the service object to gain access to these very methods create users and get all users as they have access to db

When a POST request comes to the controller to create an object,

- the controller calls the service layer,
- which uses the repository (which talks to the database),
- to save the object.

Similarly, when a GET request comes to the controller to fetch data,

- the controller again calls the service layer,
- which uses the repository to retrieve data from the database.

**Service Layer**

```
@Override
    public UserDto createUser(UserDto dto,String password)
    {
//        we have to do 3 things
//        1.convert the given userDto into user entity via the Mapper
method
//        Inside UserMapper class the mapper method takes the user
object and attaches the password to it
//        and creates the User Entity object via the Lombok's builder
method
//        2.after it has been converted to User entity save it in DB
using UserRepository
//        Thus the entire User Entity id(generated by JPA),name ,email
and pw given by User is inside the User object and saved to DB
//        3.Then Convert the saved User object into a UserDto again to
send back to the Controller Layer
//        this UserDto object will contain everything but the password
//        4.
        User user=UserMapper.toEntity(dto,password);
        User saved=userRepository.save(user);
        return UserMapper.toDto(saved);
    }
```

**DTOs**

When a GET request is made to fetch user data,
we **return a UserDto** to the client — not the full User entity — to avoid exposing sensitive or unnecessary fields.

When a POST request is made to create a user,
the client sends a **UserDto (safe public data like name, email)** and **a password (maybe as a query param)**.
We **combine** these, convert them into a full User entity (which has password, id, etc.),
and then **save it** to the database via the repository.
After saving, we **convert the entity back into a UserDto** and return it in the response.

### STEP 1: Convert DTO → Entity

Convert the incoming UserDto (which contains safe public data like name and email) into a full User entity object by using the UserMapper.toEntity(...) method.
This method also **attaches the password**, which was passed separately.

- DTOs usually **don't contain passwords** (for security).
- We **combine** both parts of the data to create the full `User` entity that can be saved in the database.
- This respects **data encapsulation** and **layer separation**.

### STEP 2: Save the Entity to the Database

Pass the User entity to the repository's .save(...) method.
Spring Data JPA handles saving this object into the database.
After saving, the User object now includes a **generated id** (like 1L) assigned by the database.

- `userRepository.save(user)` will either **insert a new row** (if it's a new user) or **update an existing one** (if it had an `id`).
- Spring JPA handles this automatically, using Hibernate under the hood.

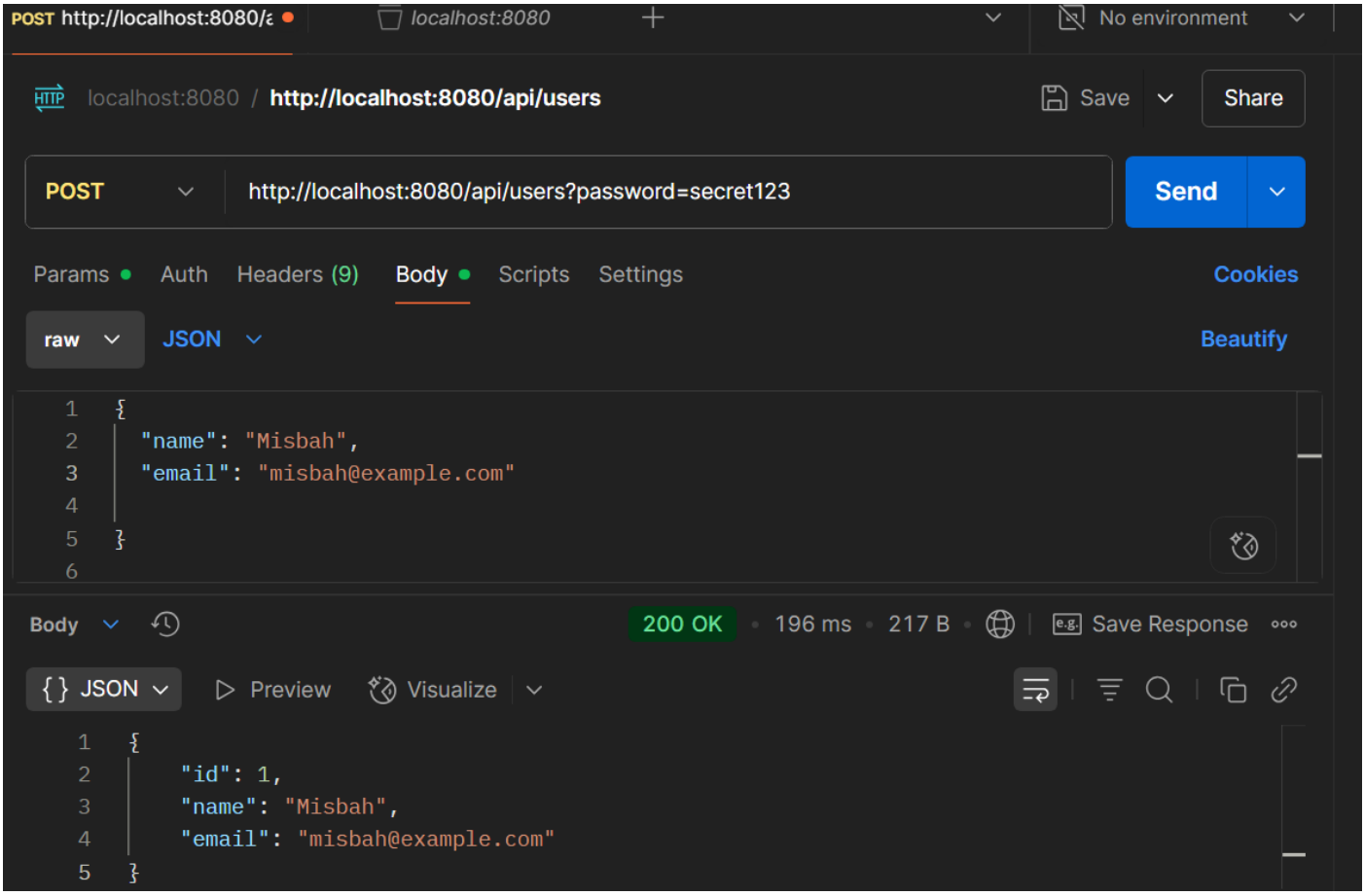
### STEP 3: Convert Entity → DTO (for response)

Convert the newly saved User entity back into a UserDto using UserMapper.toDto(...).
This will **exclude sensitive fields** like the password.

- Never expose your entity directly to the outside world (e.g., client/browser).
- Instead, return a **safe, trimmed-down DTO** that only includes necessary info like `name`, `email`, `id`.

We return the UserDto back to the controller so that it can **send a response to the client** — the user (e.g., Postman, frontend, mobile app).
It's the **response body** of the HTTP POST request.
Without this return, the client won't get any confirmation or data about what was created.

and if we don't return anything then Client gets an empty HTTP 200 OK response.

They won’t know what was saved, They can't get the new id, name, etc.

According to **REST API principles**, when you POST to create something, the server should respond with 201 Created status and A representation of the created object (usually a DTO)